Candidate Improvement to the Round-Six Exact Fourier Tensor Network

Experimental research note — independent review required

8 October 2026

Research note — October 8, 2026. This is a computationally audited modification of the finite network described in the public research draft https://github.com/shea256/fourier-transform-below-nlogn/blob/master/manuscript.md. It is not an independently reviewed Fourier theorem, an integer-multiplication algorithm, or an assertion of priority. The original NStar3 producer is attributed to Swapnil Jain and predecessors as explained in the manuscript.

Result in brief

For h=24, v=2024, m=h^2=576, replace the retained basis `(E_0,...,E_22,A)` by `(E_0,...,E_23)` in the pinned finite addition DAG, re-prune unused DAG nodes, and recompile one carrier per directed use. Keep the original signed output-piece list and the two-stage scalar/phase schedule. The new readout is

```
R_S = (sum_{j=0}^{23} E_j)/21 - (sum_{j in S} E_j)/2.
```

The identity follows because each input triple is excluded from 21 of the 24 `E_j`, so `sum E_j = 21 A` for fresh inputs. The reverse-cleanup identity works for arbitrary dirty scratch because `J L V = I`, regardless of the old scratch values.

Verified finite statistic	Original	Alternative
Active addition nodes	28,944	**28,941**
Signed output uses	20,240	20,240
Retained output uses	24	24
Auxiliary roles per local invocation R	49,208	**49,205**
Retained copy-label rank total ell	553	**552**
Simultaneous stream count W	207,387,136	**207,374,992**
Weighted sum of recursive widths s	119,453,132,304	**119,446,133,312**
Rank slack Wm-s	1,858,032	**1,862,080**
Approximate critical moment root a*	0.0000738611138603	**0.0000740260124419**

Exact-rational certificate for the alternative histogram:

```
a = 37013/500000000 = 0.000074026
theta = 1-a
Psi(theta) = sum_r n_r/W * (r/576)^theta < 1
1 - [rigorous rational upper bound on Psi(theta)] ~ 2.620588477065758e-12 > 0.
```

The certificate uses rational upper bounds for ln(576/r) from a positive series and rational upper bounds for exp(a ln(576/r)). These are *upper* bounds, so the positive gap really implies the moment inequality for the new histogram; the decimal is only a display of the exact fraction's value.

If the original manuscript's remaining tensor/array/DFT transfer arguments apply, this gives the **conditional** pure-power exponent

```
T_DFT(n) = O(n (log n)^(1-delta)), delta=0.000074025,
```

because `delta < a` leaves an exponent gap to absorb the manuscript's loglog factor. Compared with its headline `delta=0.000073`, the proposed pure-power saving increases by about 1.4%. There is *no new bit-complexity claim* for integer multiplication.

Why the modification is legitimate at the finite scalar level

Let `x_T` index source triples and define

```
E_j(x) = sum_{T: j notin T} x_T,
A(x) = sum_T x_T.
```

Every `T` has size 3, hence

```
sum_j E_j(x) = (24-3) A(x) = 21 A(x).
```

Therefore the previous readout `A - (sum_{j in S} E_j)/2` equals the alternative formula for every fresh source vector `x`. The signed piece contributions are unchanged, so the finite coefficient matrix still equals the identity. A reversible shear written as a difference of two `J L` evaluations continues to cancel arbitrary dirty scratch, because only the linear relation `J L V = I` is needed; the new `J` is linear with rational coefficients.

No new physical role is required for calculating `A` at readout: it is represented as a linear combination of the 24 already retained totals, using scalar additions and constant multiplications at the existing common readout frame (zero). These add a *fixed* number of local scalar operations and do not change asymptotic recursion weights. The global producer DAG node for `A` is no longer a retained sink; re-pruning removes three addition nodes. The omitted exclusion total `E_23` already exists in the *unpruned* DAG, and all active dependencies are recompiled explicitly.

Finite audits actually run

1. **Baseline reproducibility**. A separately reconstructed producer/compiler and binary projector replay reproduced the manuscript's *entire* original 25-bin recursive histogram, not merely the weighted sum. Baseline arithmetic and roles matched exactly.
2. **Alternate scalar coefficient test**. Exhaustively checked all `2024*2024=4,096,576` source–target coefficients in exact integer arithmetic.
3. **Full phase-frame replay**. The two local gate orientations were replayed for all physical roles; all residual edges had nested projectors and either rank zero or a nonalternating residual. Both orientations generated identical internal histograms and the correct paid retained-copy count. The alternative replay examined 1,044,282 role-transition events.
4. **Role accounting**. Verified `sum_r r H_r = h R + ell` for the new internal histogram; added the exterior edges, data fronts, interstage edges, and paid endpoint corrections without changing their algebra.
5. **Dirty-scratch scalar test**. Executed forward and inverse compiled mixers and both readouts on arbitrary integer scratch vectors at h=8 and h=24; checked the exact shear effect and restoration of every scratch slot.
6. **Exact moment test**. Strict positive upper-bound margin at `a=0.000074026`; a larger trial at `a=0.000075` fails, as expected.

The checks do **not** constitute a formal proof of the new uniform array algorithm or the DFT all-length transfer. Some replay methods are independent implementations of the upstream logic but may share its conceptual blind spots. Complete review should focus on the altered readout schedule and copied terminal carriers, in addition to the original manuscript's open proof obligations.

Entire alternative recursive histogram

width r	count n_r	width r	count n_r
1	184831680	14	14864256
2	107199136	15	4857600
3	50583808	16	12994080
4	53470032	17	10200960
5	33031680	18	33371712
6	27615456	19	13601280

width r	count n_r	width r	count n_r
7	14767104	20	22765952
8	20183328	22	44042240
9	6217728	23	16483456
10	14119424	529	8193152
11	5051904	552	199181840
12	13742960		
13	4663296		

Unlisted widths have multiplicity zero. Each child width is strictly less than 576.

Further strategies explored

- **Ground set variation**: At h=18, 20, 22, corresponding alternative roots were approximately `0.0000598689`, `0.0000694073`, and `0.0000733503`; all worse than h=24. The h=26 trial did not complete within the allotted computation limit and is not reported as a negative or positive result.
- **Producer cutoff variation**: For h=24, cutoffs 3, 4, and 5 produce the same 28,944-addition *original* producer, so this particular change has no effect. Some larger cutoff trials were computationally expensive and inconclusive.
- **Addition parenthesization**: Tested balanced, right-balanced, thirds, skewed, and modulated trees. None produced fewer active roles than the alternative's 49,205 roles in the resulting scalar DAG. A lower role count is not on its own sufficient to certify a better moment: the frame histogram must also be checked.
- **Potential next gain**: Reduce the 20,240 output-piece uses while preserving exact local reconstruction *and* admissible target-frame labels. This is a much more promising direction than re-tightening a numerical certificate, although no concrete new saving was constructed here.
- **Rank-of-readout obstruction**: With the present fixed signed piece outputs, the coefficient vectors `w_S=(1/21)1-(1/2)1_S` span dimension 24: their differences span the 23-dimensional sum-zero subspace, and their common nonzero sum component supplies the remaining direction. Thus simply dropping a retained total without modifying the piece decomposition cannot recover all readouts. Further basis reductions require a more substantive circuit change.

Reproduction

Save these files in the same folder and run:

```
python3 fft_new_retained_basis_audit.py
python3 fft_modified_scalar_check.py
```

The first script fully rebuilds the producer, active DAG, compiled physical carrier graph, binary projectors, physical residual histogram, and exact rational certificate. It checks against the manuscript's original histogram before printing the new result. The second performs direct dirty-scratch scalar trials. Standard-library Python; CPU and memory requirements are substantial but finite.

Research and attribution boundaries

Original public draft: https://github.com/shea256/fourier-transform-below-nlogn/blob/master/manuscript.md

Attributed finite producer: Swapnil Jain, pinned `f2176bc1124821bf17eb63725bd366d7bdc020a3`; see the draft's attribution to preceding contributors. The local port is a modification, not the exact vendored upstream byte sequence. See the original repository's Apache-2.0 license and NOTICE. New basis, recompilation, audits and candidate calculations were developed in this conversation with AI assistance and have not been independently refereed. No claim is made that the modification was unknown to other researchers.